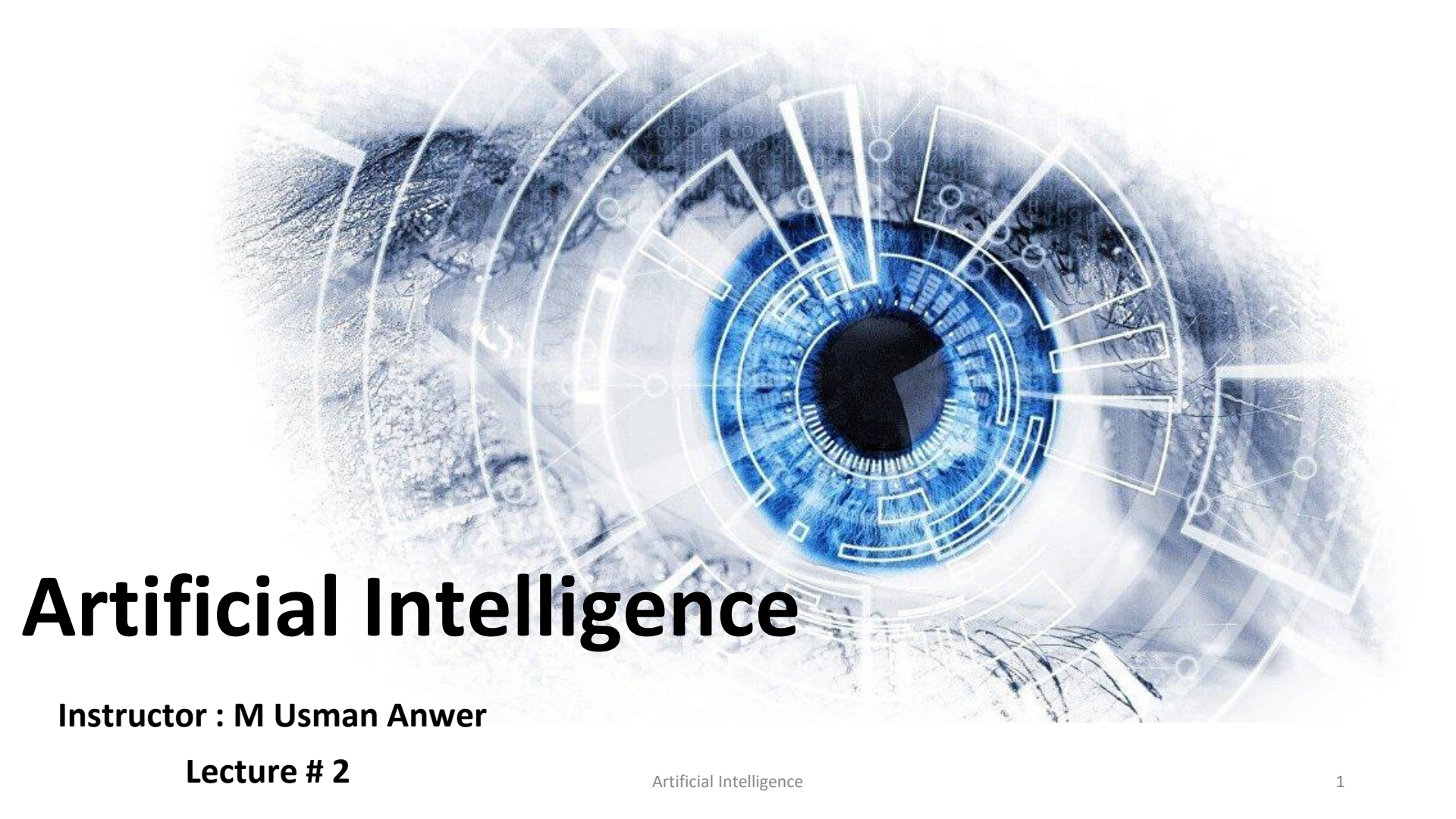




Artificial Intelligence

Instructor : M Usman Anwer

Lecture # 2

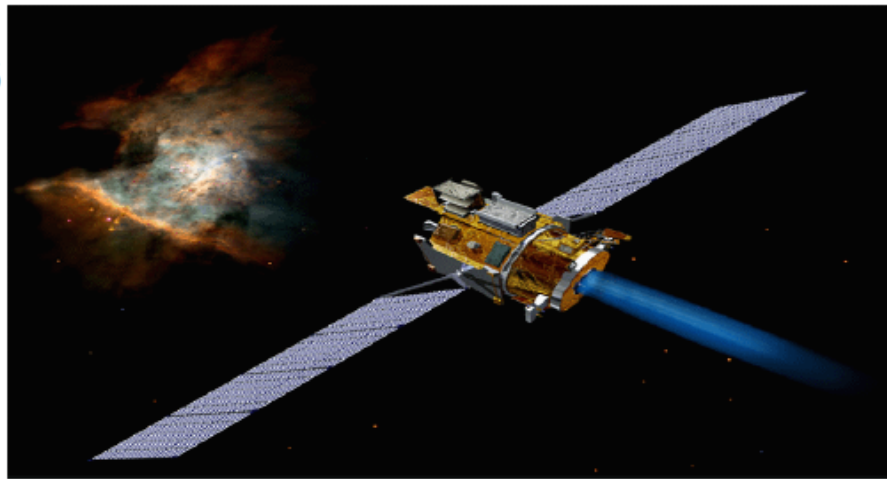
Autonomous Agents for specialized tasks

- Agents (and their physical instantiation in robots) have a role to play in high-risk situations, unsuitable or impossible for humans
- The degree of autonomy will differ depending on the situation (remote human control may be an alternative, but not always)
- Examples:
 - Air Traffic Control
 - Internet Agents
 - Customer service
 - Atomic Reactor

Example: Space Craft Agents

This is not fiction: NASA's DS1 has done it!

- <http://nmp.jpl.nasa.gov/ds1/>
- **“Deep Space 1**
launched from Cape Canaveral on October 24, 1998. During a highly successful primary mission, it tested 12 advanced, high-risk technologies in space. In an extremely successful extended mission, it encountered comet Borrelly and returned the best images and other science data ever from a comet. During its fully successful hyperextended mission, it conducted further technology tests. The spacecraft was retired on December 18, 2001.” – NASA Web site



The Structure of Intelligent Agents

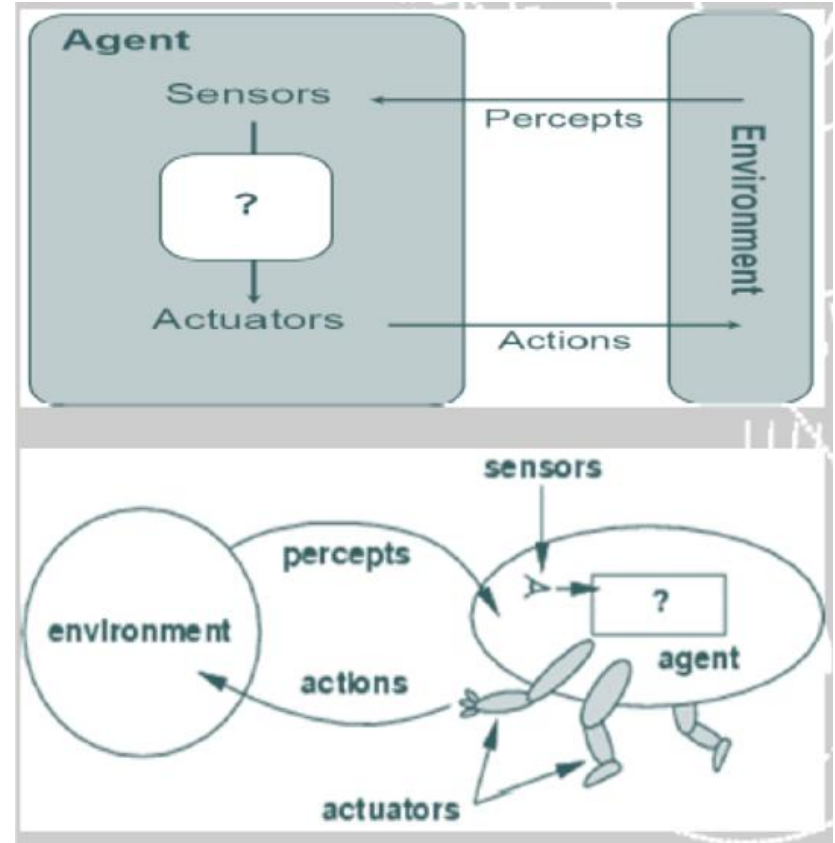
Agent = Architecture + Agent Program

- Architecture = the machinery that an agent executes on.
- Agent Program = an implementation of an agent function.

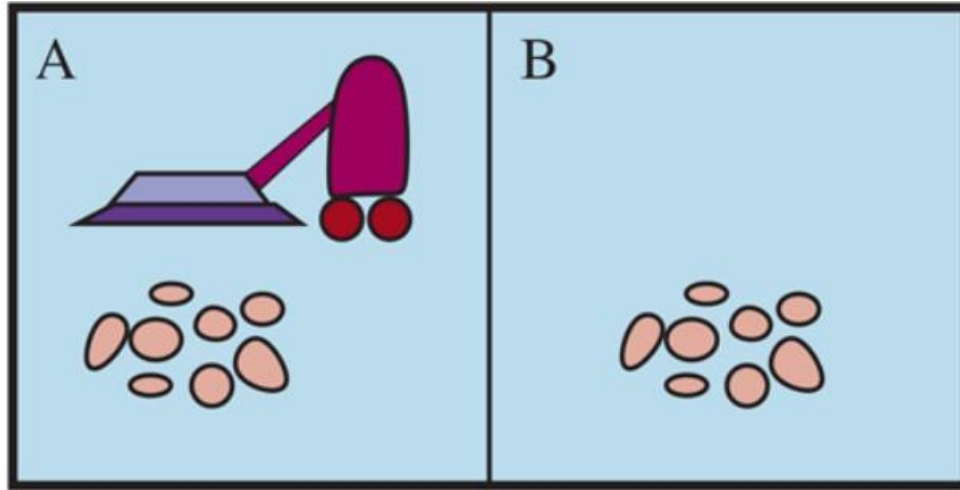
Agent



- Human Agent
- Hardware/ Robotic Agent
- Software Agent
- The problem the agent solves is characterized by **(PEAS)**
Performance Measure,
Environment, Actuators, and
Sensors.



PROBLEM SOLVING AGENTS



A vacuum-cleaner world with just two locations. Each location can be clean or dirty, and the agent can move left or right and can clean the square that it occupies. Different versions of the vacuum world allow for different rules about what the agent can perceive, whether its actions always succeed, and so on.

PEAS

Performance	Environment	Actuator	Sensors
• Clean all squares (A and B) • Minimize number of actions • Minimize time taken • Minimize energy consumption • Avoid unnecessary movement	Two locations: A and B Each location can be Clean or Dirty Vacuum agent is located in either A or B Environment may be: • Deterministic • Fully observable • Static • Discrete • Single-agent	The agent can perform the following actions: Suck → Clean the current location Move Left → Move from B to A Move Right → Move from A to B NoOp → Do nothing (when everything is clean)	The agent can sense: Current location (A or B) Current location's status (Clean or Dirty)

Environment Types

- **1. Fully Observable vs Partially Observable**

- An agent's sensors give it access to the complete state of the environment at each point in time else only partial information of environment is known.

- **2. Deterministic vs. Stochastic**

- **Deterministic:** The next state of the environment is completely determined by the current state and the action executed by the agent
- **Stochastic (Non-Deterministic):** There can be more than one next state, for a given state-action combination
 - Taxi Driving is clearly stochastic

Environment Types

- **3. Static vs Dynamic**

- The environment is unchanged while an agent is deliberating which action to execute

- **4. Discrete vs Continuous**

- The environment is discrete if the number of actions and possible states of the environment is finite otherwise it is continuous.

- **5. Single Agent vs Multi-Agent**

- In the multi-agent case, the performance measure of one agent depends on the performance measures of the other agent(s)
- **Competitive multi-agent:** Chess Playing
- **Collaborative multi-agent:** Robo Soccer.
- A quite complicated field which is currently the focus of much research.

Driver Example

Performance Measure	Environment	Actuators	Sensors
safe, fast, legal, comfortable trip, maximize profits	roads, other traffic, pedestrians, customers	steering, accelerator, brake, signal, horn, display	camera, sonar, speedometer, GPS, odometer, engine sensors, keyboard, accelerator

DARPA urban challenge 07:

<http://www.youtube.com/watch?v=SQFEmR50HAK>

Medical Diagnosis System

Performance Measure	Environment	Actuators	Sensors
healthy patient, minimize costs, lawsuits	patient, hospital, staff	display questions, tests, diagnosis, treatments, referrals	keyboard entry of symptoms, findings, patient's answers



Mushroom-Picking Robot

Performance Measure	Environment	Actuators	Sensors
Percentage of good mushrooms in correct bins	Conveyor belt with mushrooms, bins	Jointed arm and hand	camera, joint angle sensors



Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
<i>Crossword puzzle</i>	fully	deterministic	sequential	static	discrete	single
<i>Chess with a clock</i>	fully	strategic	sequential	semi	discrete	multi
<i>Taxi driver</i>	partially	stochastic	sequential	dynamic	conti.	multi
<i>mushroom-picking</i>	partially	stochastic	episodic	dynamic	conti.	single

- The environment type largely determines the agent design
- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Activity

Develop PEAS description for the following task environment:

- **Robot soccer player**
- **Shopping for used AI books on the Internet**

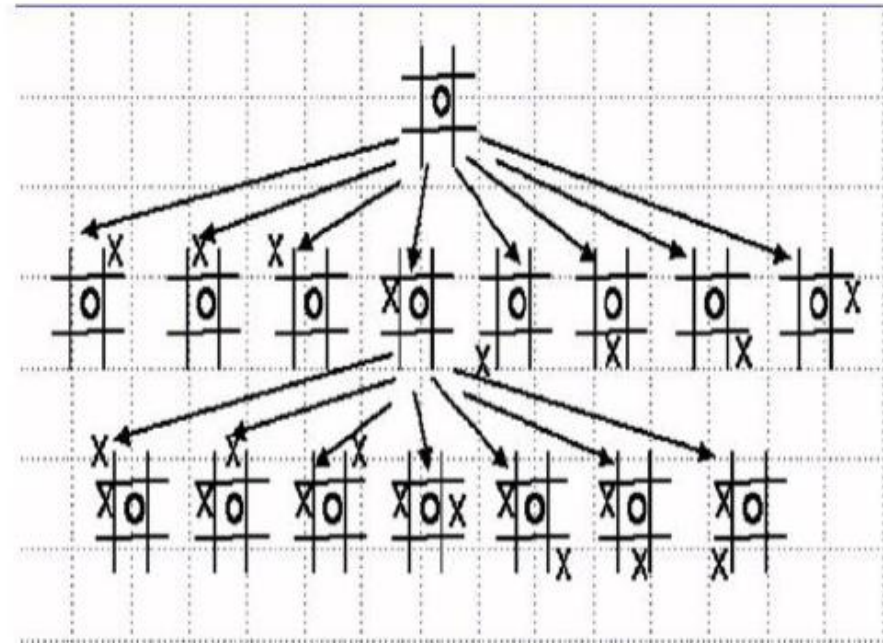
Problem Solving - Elements of problem

- 1. Initial Configuration / state
 - Data representation by Nodes, Edges
- 2. Operators
 - decisions/functions
- 3. Goal Test
 - test function
- 4. Best solution selection
 - Cost function

State Space Modeling

Agents and problem solving

- Intelligent agents can solve problems by searching a state-space
- State-space Model
 - the agent's model of the world
 - usually a set of discrete states
 - e.g., in driving, the states in the model could be towns/cities
- Goal State(s)
 - a goal is defined as a desirable state for an agent
 - there may be many states which satisfy the goal test
 - e.g., drive to a town with a ski-resort
 - or just one state which satisfies the goal
 - e.g., drive to Mammoth
- Operators (actions, successor function)
 - operators are legal actions which the agent can take to move from one state to another



- Environment is static
 - no changes in environment while problem is being solved
- Environment is observable
- Environment and actions are discrete
 - (typically assumed, but exceptions are always there)
- Environment is deterministic

Downloaded from <http://ajph.org/> on November 10, 2015

- Individual cities

- ## Operators

- Freeway routes from one city to another

- Freeway routes from one city to another

- **Current City**

- **Current City**

- Set of cities we would like to be in

- Set of cities we would like to be in

- A specific goal city,

- A specific goal city,

- A sequence of operators which get us there,



Example (1/2)

- Suppose we have an agent in the City of Arad, Romania enjoying a touring holiday.
- With performance measures containing many factors:
 - Improve suntan
 - enjoy the nightlife (such as it is),
 - avoid hangovers
 -
- The decision problem is a complex one with many tradeoffs.

Example (2/2)

- Now suppose the agent has a nonrefundable ticket to fly out of Bucharest the following day.
- Than it makes sense for the agent to adopt the **goal of getting to Bucharest**.
- By adopting this goal,
 - The agent rejects the actions that don't reach Bucharest on time.
 - Thus, the agent's decision problem is greatly simplified.

Road map of Romania

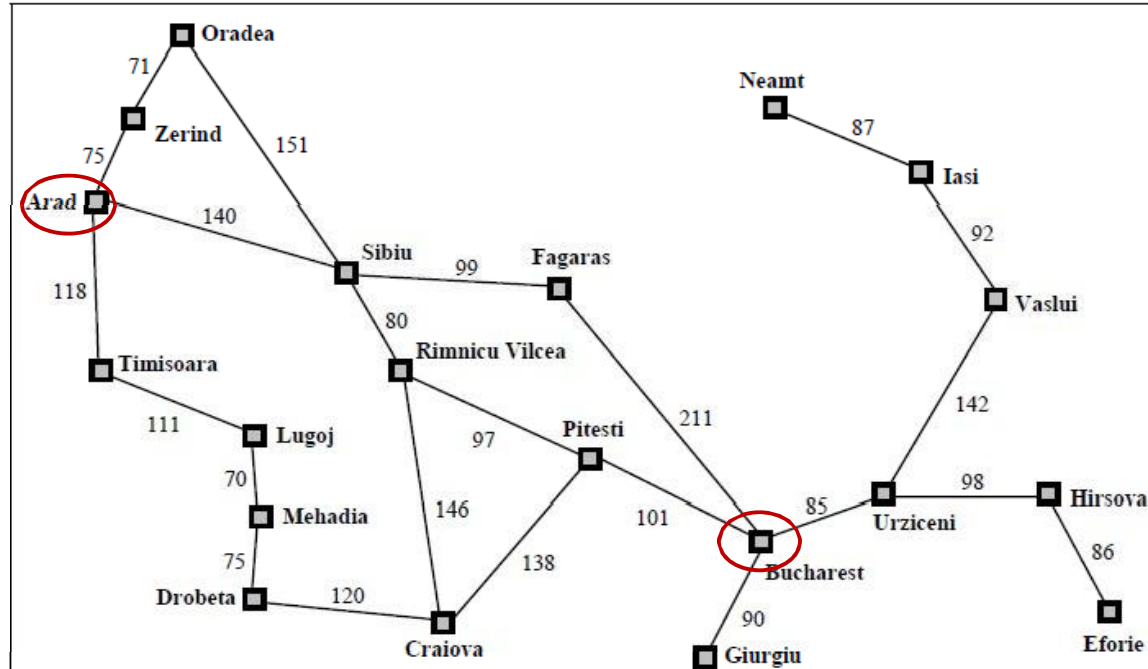


Figure 3.2 A simplified road map of part of Romania.

Goal formulation

- Goals help organize behavior by limiting the objectives that the agent is trying to achieve.
- The agent's task is to find out which sequence of actions will get it to a goalstate
- **Goal formulation** , based on the current situation and the agent's performance measure, is the first step in problem solving.

Problem formulation

- **Problem formulation:** the process of deciding what actions and states to consider, given a goal.
- in the example
 - actions: driving from one major town to another
 - states: being in a particular town

“Formulate, Search, Execute”

- The process of looking for a **sequence** that leads to a goal state is called **search**.
- A search algorithm takes a **problem** as input and returns a **solution** in the form of an action sequence.
- Once a solution is found, the actions it recommends can be carried out. This is called the **execution** phase.
- Thus, we have a simple “formulate, search, execute” design for the agent.

Assumptions for environment

- Assumptions for the **environment** of agent design in are:
- **Static**: without paying attention to any changes that might be occurring in the environment.
- **Observable** “knows the initial state”
- **Discrete**: “alternative courses of action” can be enumerated
- **Deterministic**: No unexpected events
- All these assumptions mean that we are dealing with the easiest kinds of environments.

Well defined problems and solutions

- **Goal formulation** is the first step in problem-solving.
- Then we need the **problem formulation** which is defined by **four components**:
 1. -Initial state
 2. -Successor function (or operator)
 3. -Goal test
 4. -Path cost

Components of a problem formulation

1-Initial state

The state that the agent starts in. -ex: in(Arad)

2-Successor function

-Given a particular state x *SUCCESSOR-FN*(x) returns a set of $\langle \text{action}, \text{successor} \rangle$ ordered pairs

-Ex: From state in(Arad) successor function returns:
 $\{ \langle \text{Go(Sibiu)}, \text{In(Sibiu)} \rangle, \langle \text{Go(Zerind)}, \text{In(Zerind)} \rangle, \langle \text{Go(Timisoara)}, \text{In(Timisoara)} \rangle \}$

Components of a problem formulation

3-Goal test -Determines whether a given state x is a goal state.

“ $x = \text{in(Bucharest)}$ ”

Goal states can be:

-explicit “ in(Bucharest) ”

4-Path cost -A function that assigns a numeric cost to each path.

-Sum of **step costs** $c(x, a, y) > 0$

State space, path

- Together, the initial state and successor function implicitly define the **state space** of the problem
 - the set of all states reachable from the initial state.
- The state-space forms a graph in which the **nodes** are states and the **arcs** between nodes are actions.
- A **path** in the state space is a sequence of states connected by sequences of actions.

State space graph

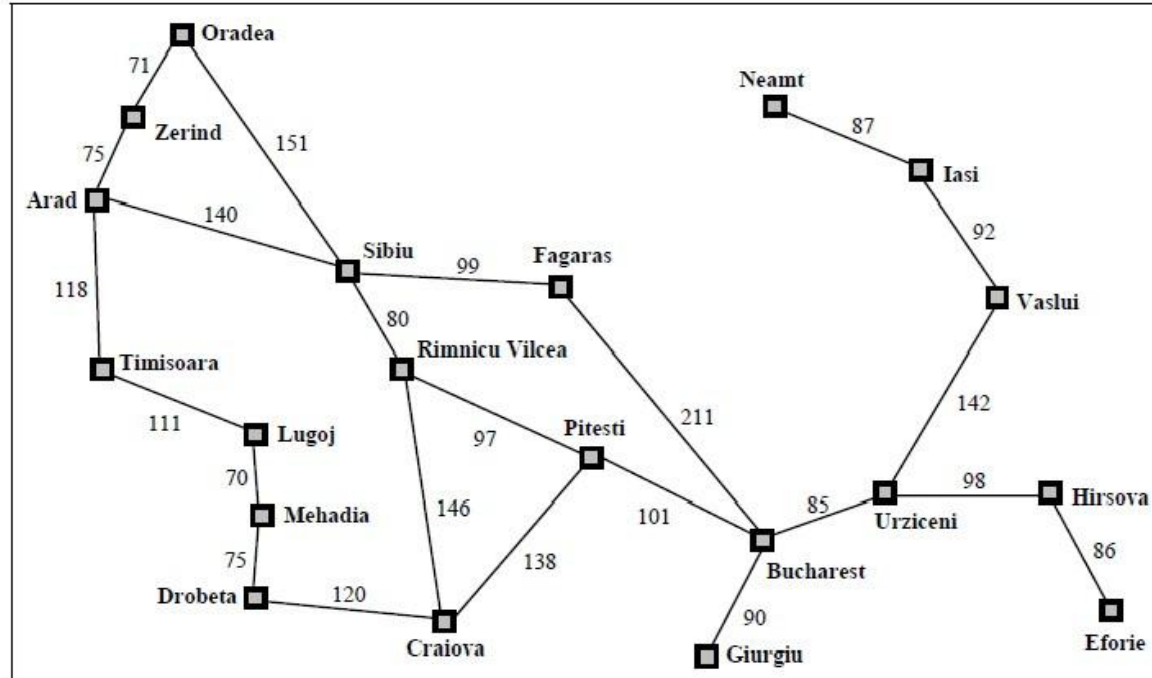


Figure 3.2 A simplified road map of part of Romania.

Solution quality, optimal solution

- A **solution** to a problem is a path from the initial state to a goal state.
- **Solution quality** is measured by the path cost function and an **optimal solution** has the lowest path cost among all solutions.

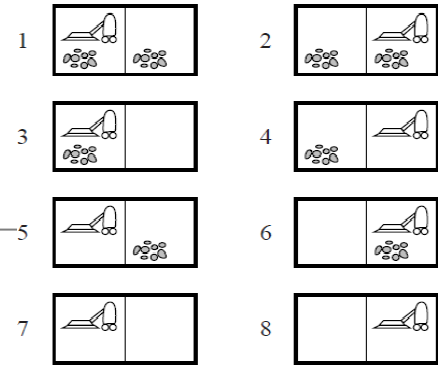
Formulating problems

- We previously proposed a formulation of the problem getting to Bucharest in terms of the initial state, successor function, goal test, and path cost.
- This formulation seems reasonable, yet it omits a great many aspects of the realworld.
- A state of the real world includes **more detail** than the simple state description “in(Bucharest)”.
(details like traveling companions, condition of the road, what is on the radio, the weather, etc.)

EXAMPLE PROBLEMS

- **Toy problem**
 - Intended to illustrate or exercise various problem-solving methods.
 - Concise, exact description
 - Can be used to compare the performance of algorithms
- **Real-world problem**
 - Whose solutions do people actually care about
 - They tend not to have a single agreed-upon description

Some toy problems

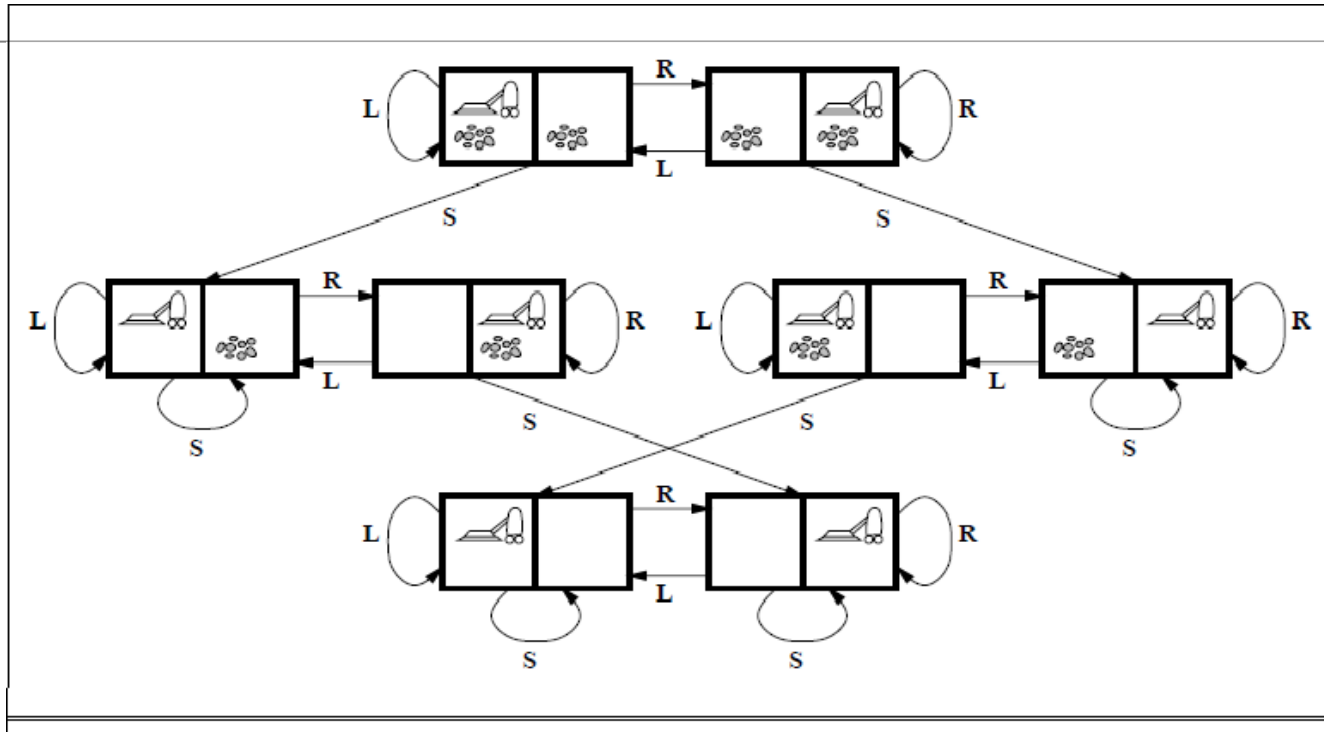


Vacuum World

2×2^2

- States: location of the agent, containing dirt or not possible world states.
- Initial state: any state
- Successor function: legal states that result from three actions (Left Right and Suck)
- Goal state: checks whether all the squares are clean.
- Path cost: # of steps in the path (each step costs 1)

Some toy problems



8-Puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- Consist of a 3x3 board with 8 numbered tiles and a blank space.
- Tile adjacent to the blank space can slide into the space
- Object: reach a specified goalstate

Formulation of 8-Puzzle

- **State space:** location of each of the eight tiles and the blank
- **Initial state:** Any state
- **Successor function:** states resulting from trying the four actions(blank moves left, right, up, or down)
- **Path cost:** number of steps in the path

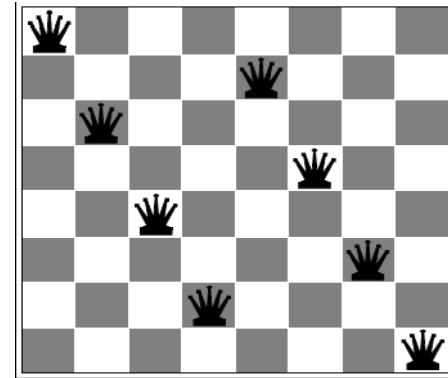
- 8-puzzle belongs to the family of sliding block puzzles
- This general class is known to be NP-complete
- $9!/2=181,440$ reachable states and easily solved

8-Queens Problem

- Place 8-queens on a chessboard such that no queen attacks any other.
- Two main kinds of formulation

1- Complete-state formulation

Starts with all 8 queens on the board and moves them around.



8-Queens Problem

2-Incremental formulation:

- States:** Any arrangement of 0 to 8 queens on the board
- Initial state:** no queens on the board
- Successor function:** Add a queen to any empty square
- Goal test:** 8 queens are on the board, none attacked.

8-Queens Problem

- Prohibit placing a queen in any square that is already attacked (a better formulation)
 - **States:** Arrangements of n queens ($0 \leq n \leq 8$), one per column in the leftmost n columns, with no queen attacking another
 - **Successor function:** add a queen to any square in the leftmost empty column such that it is not attacked by any other queen (Reduces state space from 3×10^4 to just 2,057)

Real world problems

- Some of the real-world problems are,
 - Route finding problem
 - Touring problems
 - Traveling salesperson problem
 - VLSI layout problem
 - Robot navigation
 - Automatic assembly sequencing
 - Internet searching

Route finding problem

- Used in a variety of applications
 - routing in computer networks, military operations planning, airline travel planning systems, etc.
- Typically, complex to specify

Route finding problem

- A simplified example of an airline travel problem:
 - States:** Location (e.g., an airport) and the current time
 - Initial state:** Specified by the problem
 - Successor function:** States resulting from taking any scheduled flight (seat class and location), leaving later than the current time.
 - Goal test:** Are we at the destination by some prespecified time?
 - Path cost:** Depends on monetary cost, waiting time, flight time, seat quality, and so on.
 - many additional complications to handling by zantine structure that airlines impose
 - Contingency plans should also be included

Touring problems

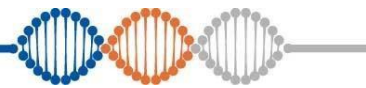
- Closely related to route-finding problems
- Ex: “**Visit every city at least once, starting and ending at Bucharest**”
- Important differences,
 - **States:** “current location, set of cities the agent has visited”
 - Ex: in Vaslui; visited{Bucharest, Urziceni, Vaslui}
 - **Initial state:** “In Bucharest; visited(Bucharest)”
 - **Goal test:** -Is the agent in Bucharest?
 - Visited all cities?

Traveling salesperson problem (TSP)

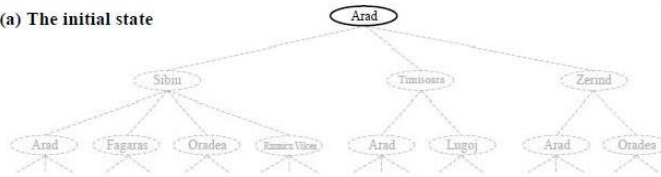
- A touring problem in which
 - Each city must be visited **exactly once**
- Aim: Find the shortest tour
- Known to be NP-hard

SEARCHING FOR SOLUTIONS

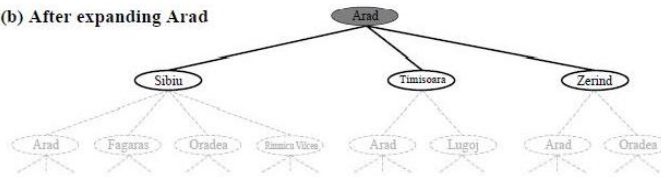
- Having formulated some problems, we now need to solve them.
- This is done by a search through the state space.
- This chapter deals with some search techniques that use an explicit **search tree**.



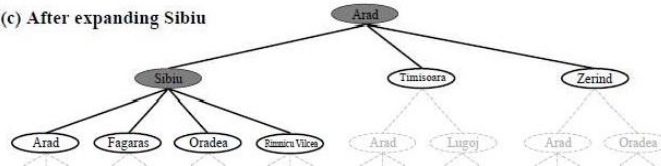
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



Partial
search trees

function TREE-SEARCH(*problem*) **returns** a solution, or failure
initialize the frontier using the initial state of *problem*

loop do

if the frontier is empty **then return** failure

choose a leaf node and remove it from the frontier

if the node contains a goal state **then return** the corresponding solution
expand the chosen node, adding the resulting nodes to the frontier

Problem: find a route from Arad to Bucharest.

-Expanding : Applying successor function to the current state, thereby generating a new set of states.

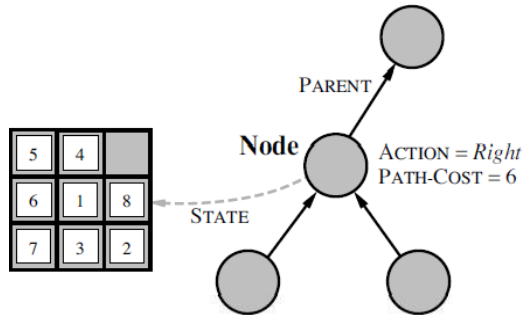
-The choice of which state to expand is determined by the **search strategy**.

-When a goal state found, the solution is sequence of actions from goal node back to the root.

-Frontier: a queue that stores the set of nodes which are generated but not yet expanded

Representation of Nodes

- A node is a data structure with five components
 - **State:** the state in the state space that the node corresponds
 - **Parent-node:** the node in the search tree that generated this node
 - **Action:** the action that was applied to the parent to generate the node
 - **Path-cost:** cost of the path from the initial state to the node
 - **Depth:** The number of steps along the path from the root.



-**Node:** bookkeeping data structure to represent a search tree

-**State:** corresponds to

Figure 3.10 Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

Measuring problem-solving performance

Evaluation of an algorithm's performance

- **Completeness:** Is the algorithm guaranteed to find a solution when there is one?
- **Optimality:** Does the strategy find the optimal solution
- **Time complexity:** How long does it take to find a solution?
- **Space complexity:** How much memory is needed to perform the search?

Measuring problem-solving performance

- **Time** and **space** complexity are always considered concerning some measure of the problem difficulty.
- In AI, complexity is expressed in terms of three quantities:
 - B, the branching factor, max number of successors at any node
 - D, the depth of the shallowest goal node
 - M, the maximum length of any path in the state space
- **Time** is often measured in terms of several **nodes generated** during the search.
- **Space** is measured in terms of the **maximum number of nodes stored in memory**.

Assessing the effectiveness of an algorithm

- Search cost
 - depends on the time complexity
 - can also include a term for memory usage
- Total cost
 - combines the search cost and the path cost of the solution found

Conclusions

- Interesting problems can be cast as search problems, but you've got to set up state space
- Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored